Lecture 8: Introduction to Formal Methods (Cont.)

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn (link to their course)

EECS 219C: Formal Methods: Specification, Verification, and Synthesis at UC Berkeley by Sanjit Seshia

CS 357 Advanced Topics in Formal Methods at Stanford by Aleksandar Zeljic

Trustworthy AI Systems at UIUC by Gagandeep Singh and Madhu Parthasarathy

Other relevant courses on Trustworthy Machine Learning:

CS 329T at Stanford

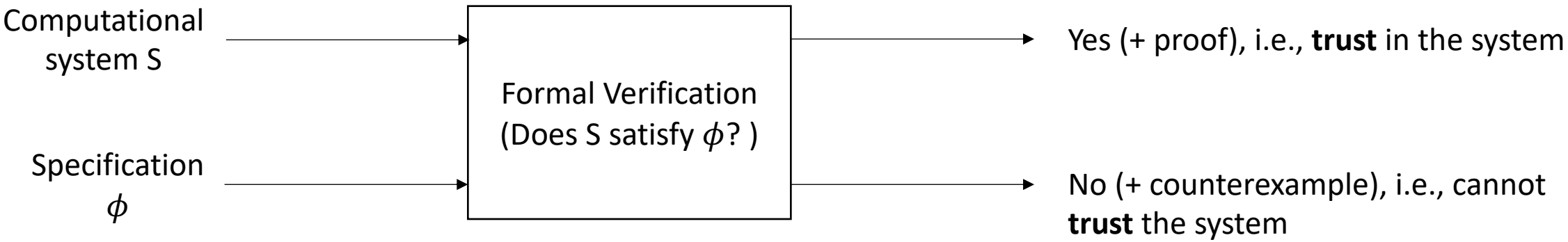
CS 521 at UIUC

ECE1784H/CSC2559H at U Toronto

Recap

- Proofs vs Formal Proofs
- Formal methods are computational proof methods for verifying and searching for formal proofs
- Applications of formal methods to computer science:
  - Formal verification
  - Synthesis
- Formal verification requires:
  - Formal specification
  - Mathematical model of the system
- Formal verification approaches frequently use SAT/SMT solvers

Recap: Formal verification



Recap: Constraint satisfaction problem (Example 1)

x, y, z : Boolean variables
Find values such that all the following constraints are satisfied

x | ~y | ~z
~x | y | ~z
~x | ~y | z

Recap: Constraint satisfaction problem (Example 2)

x, y, z : real-valued variables
Find values such that all the following constraints are satisfied

$$2x + 3y - 5z \leq 7$$
$$-5x + 2y \leq 106$$
$$9x - 4y + 29z \leq -1$$

What if we change the type to "integer" variables

Recap: Constraint satisfaction problem (Example 3)

x, y, z : real-valued variables
Find values such that all the following constraints are satisfied

$2x + 3y - 5z \leq 7 \mid 3x - 4z \leq 9$
$-5x + 2y \leq 106 \mid 6y - 78z \leq 567$
$9x - 4y + 29z \leq -1 \mid 5x - 7y + z \leq 98$

Mixture of Boolean and linear constraints

Formal verification: Example 1

Input: int x, int y
Output: int r

```
z := x + y;
z' := x – y;
r := z * z'
```

Post-condition / assertion / "ensures" (in Dafny): $r = x^2 - y^2$

Cannot define correctness without specifications (logical constraints)

Verification conditions (VCs)

Input: int x, int y
Output: int r

z := x + y;
z′ := x – y;
r := z * z′

Ensures $r = x^2 - y^2$

Verifying correctness means checking validity of the following logical formula, called VC (Verification Condition):

$(z = x+y \;\&\&\; z' = x-y \;\&\&\; r = z*z') \rightarrow r = x^2 - y^2$

From VC to constraint satisfaction

Verification problem: check validity of

$(z = x+y \;\&\&\; z' = x-y \;\&\&\; r = z*z') \rightarrow r = x^2 - y^2$

The answer is NO if the following set of constraints over integer variables are satisfiable:

$\{ z = x+y;\;\; z' = x-y;\;\;\; r = z*z';\; r \mathrel{!}= x^2 - y^2 \}$

Satisfying solution to these constraints is a "counterexample" or a bug that shows why the program is incorrect

Formal verification: Example 2

```
Input:  int[32] x,  y  /* fixed precision 32 bit integers (as in C) */
Output: int[32] r
 int[32] z := x + y;        /*what's the precise semantics of addition?? */
 r := z / 2
Ensures  (x + y = 2r)                /* r is average of x and y */
```

Verification condition: Validity over int[32] variables

$z = x + y \;\&\&\; r = z/2 \rightarrow (x + y = 2r)$

Constraint satisfaction problem :

$\{ z = x + y;\ r = z/2;\ x + y \mathrel{!}= 2r \}$

Satisfying solution: $x=y=2^{32} - 1;\ r=z=0$

Types matter!

Need to capture precise semantics of language constructs!

Formal logic for specifications

Specification is a logical formula constructed from expressions over program variables using logical operators AND &&, OR |, NOT ~

Different choices based on:

- Allowed types of variables:
  - Boolean (bool), integer (int), natural numbers (nat), real numbers (real)
  - Enumerated types
  - Bit-vectors (fixed precision integers): int[32], int[64]
  - Arrays and matrices
- Operators allowed in expressions, e.g. for integers, three classes:
  - Difference constraints: $x - y \leq 5$
  - Linear constraints: $2x + 3y - 5z \leq 7$
  - Full arithmetic: $r = x^2 - y^2$
- Whether quantifiers over variables are allowed

The more restricted the specification logic, easier it is for a tool to solve the resulting constraint satisfaction problem

Formal verification of programs with loops

```
Input:  int x1, x2
Output: int y1, y2

Requires (0 ≤ x1 && x2 > 0)     /* Assumption on inputs; also called pre-condition */
 y1 := 0;
 y2 := x1;
 while (x2 ≤ y2) {
          y1 := y1+1;
          y2 := y2 – x2
          };
Ensures  ( y1 = x1 div x2 && y2 = x1 rem x2 )
```

Formal verification of programs with loops

Current verification tools require the programmer also to specify "loop invariants"

Condition over program variables that is satisfied at the beginning of each iteration

```
Requires (0 ≤ x1 && x2 > 0)
 y1 := 0;
 y2 := x1;
 while (x2 ≤ y2) {
                    Invariant ϕ : (x1 = y1*x2 + y2 && 0 ≤ x1 && x2 > 0)
        y1 := y1+1;
        y2 := y2 – x2
        };
Ensures  ( y1 = x1 div x2 && y2 = x1 rem x2 )
```

Formal verification of programs with loops

Requires $(0 \leq x1 \;\&\&\; x2 > 0)$

```
y1 := 0;
y2 := x1;
while (x2 ≤ y2) { Invariant ϕ : (x1 = y1*x2 + y2 && 0 ≤ x1 && x2 > 0)
        y1 := y1+1;
        y2 := y2 - x2
        };
```

Ensures $(y1 = x1 \text{ div } x2 \;\&\&\; y2 = x1 \text{ rem } x2)$

Verification conditions:

1. Initialization ensures that invariant holds on first iteration
   $(0 \leq x1 \;\&\&\; x2 > 0) \rightarrow \phi[y1/0;\ y2/x1]$
2. Execution of the loops preserves loop invariant
   $(x2 \leq y2) \;\&\&\; \phi \rightarrow \phi[y1/y1+1;\ y2/y2-x2]$
3. Post-condition holds upon termination of loop
   $\sim(x2 \leq y2) \;\&\&\; \phi \rightarrow (y1 = x1 \text{ div } x2 \;\&\&\; y2 = x1 \text{ rem } x2)$

Summary of formal verification

- Proving correctness of programs requires programmer to write logical specifications
  - Pre/Post conditions for each function
  - Invariants for loops

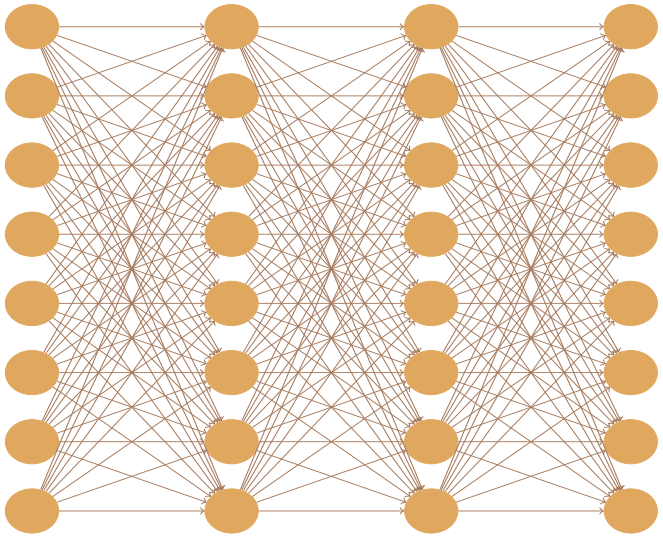
- Proving correctness can then be translated automatically to constraint solving
  - Scalable SAT solvers for constraints over Boolean variable
  - Specialized scalable SMT solvers for more general classes of constraints / variable types

- Useful references

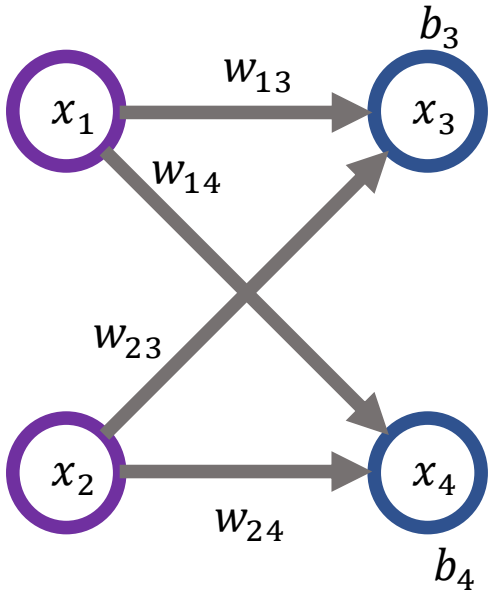
  - Satisfiability modulo theories: introduction and applications; de Moura and Bjorner; CACM 2011
  - The dogged pursuit of bug-free C programs: The Frama-C software analysis platform; Baudin et al; CACM, 2021
  - Dafny programming and verification language: https://dafny.org/
  - Talk by Byron Cook of Automated Reasoning Group at Amazong Web Services: An AWS Approach to Higher Standards of Assurance w/ Provable Security

Formal verification of neural networks

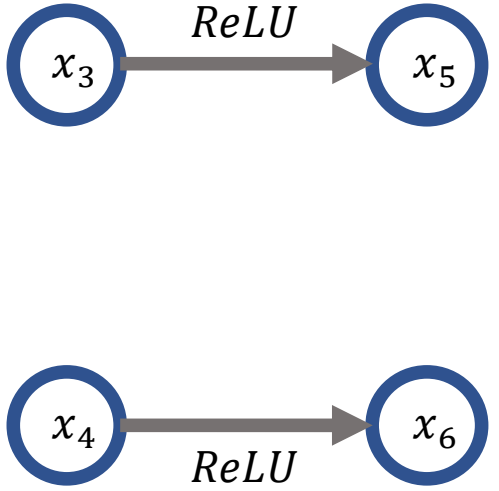
What is a deep neural network?



Each layer is a function

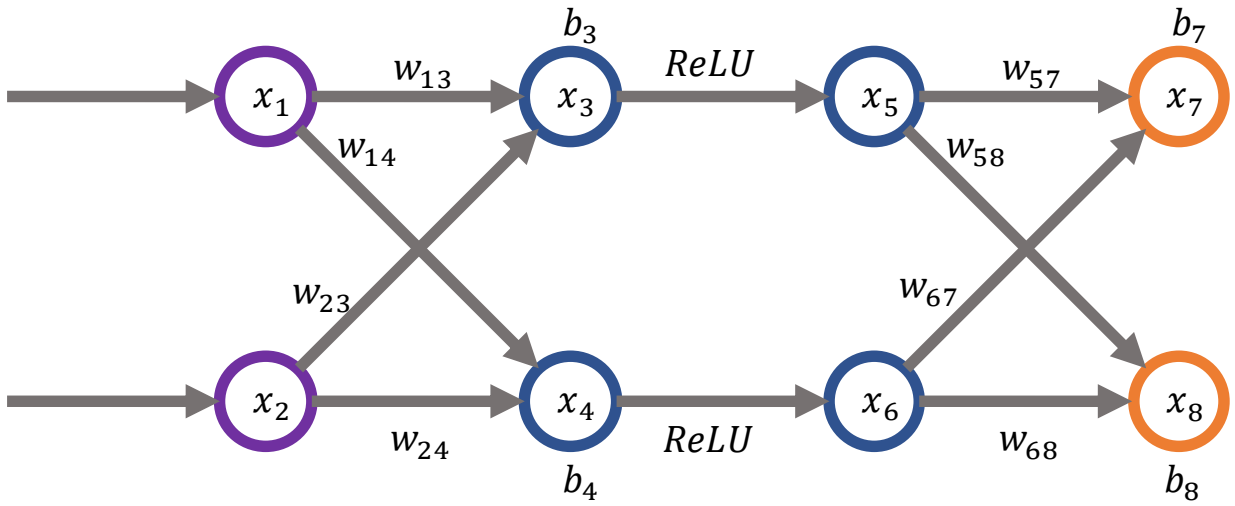


$$(x_3, x_4) \leftarrow f_1(x_1, x_2) \text{ where}$$
$$x_3 = w_{13} \cdot x_1 + w_{23} \cdot x_2 + b_3$$
$$x_4 = w_{14} \cdot x_1 + w_{24} \cdot x_2 + b_4$$



$$(x_5, x_6) \leftarrow f_2(x_3, x_4) \text{ where}$$
$$x_5 = ReLU(x_3) = \max(0, x_3)$$
$$x_6 = ReLU(x_4) = \max(0, x_4)$$

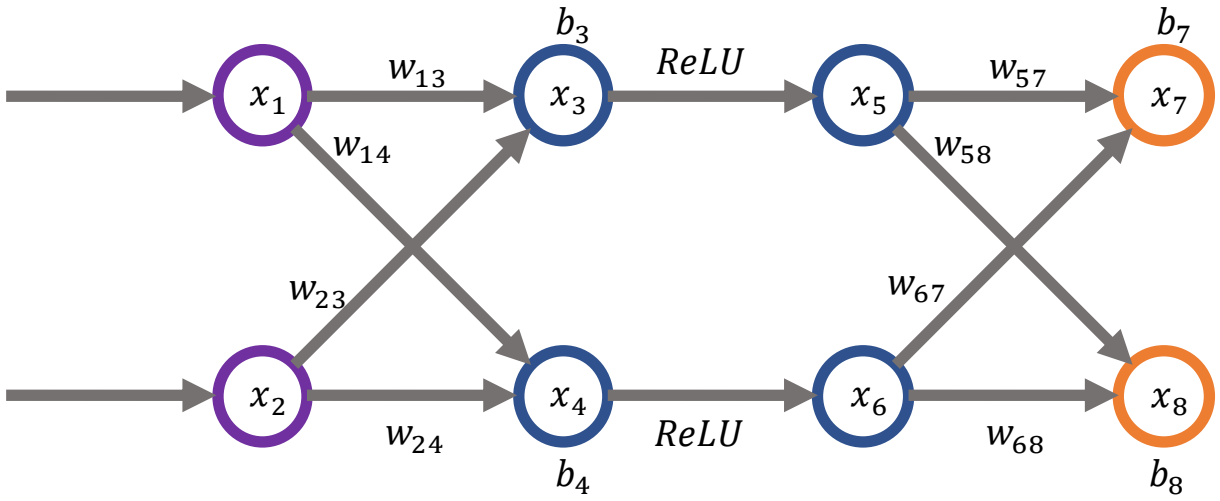
DNN is a composition of layerwise functions



$(x_7, x_8) \leftarrow f(x_1, x_2) = f_3 \circ f_2 \circ f_1(x_1, x_2)$ where f_3 is the function computed by the third layer

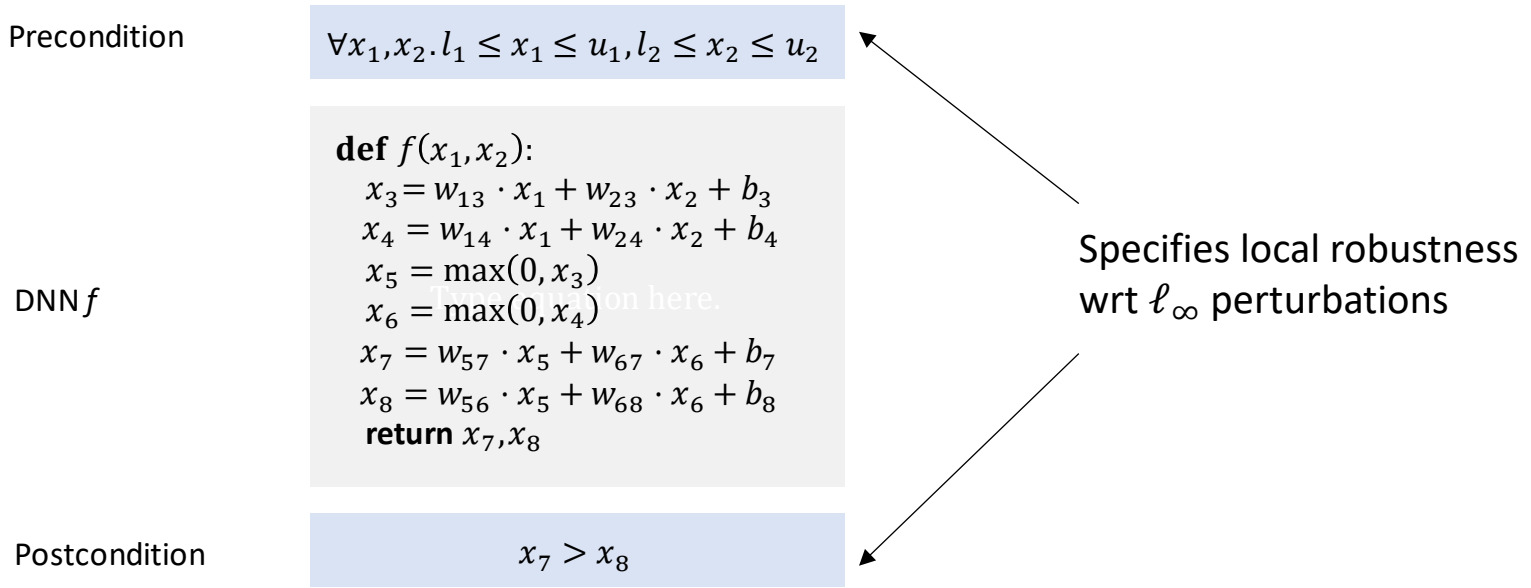
DNNs as programs

DNNs can be seen as straight-line programs (i.e., programs without loops)



def $f(x_1, x_2)$:
$x_3 = w_{13} \cdot x_1 + w_{23} \cdot x_2 + b_3$
$x_4 = w_{14} \cdot x_1 + w_{24} \cdot x_2 + b_4$
$x_5 = \max(0, x_3)$
$x_6 = \max(0, x_4)$
$x_7 = w_{57} \cdot x_5 + w_{67} \cdot x_6 + b_7$
$x_8 = w_{56} \cdot x_5 + w_{68} \cdot x_6 + b_8$
return x_7, x_8

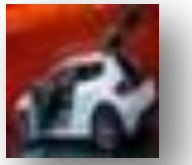
Formal specifications for DNNs



Either prove that the network output satisfies the postcondition for all inputs in the pre-condition or find a counterexample

Formal specifications for DNNs: Local robustness

I_0



Network correctly classifies I_0 as "car"

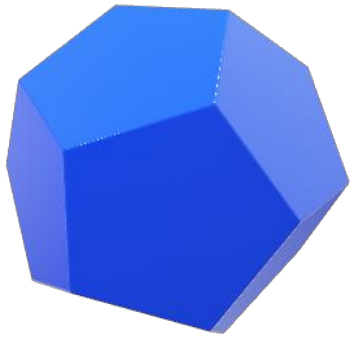
L_∞-ball around I_0 of radius ϵ

$0.6 \leq x_0 \leq 0.65$
$0.55 \leq x_1 \leq 0.6$
.....

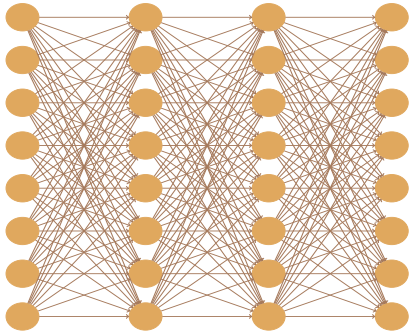
ϕ with $\epsilon = 8/255$

ψ: network classifies image as "car"

Formal specifications for DNNs: A geometric view



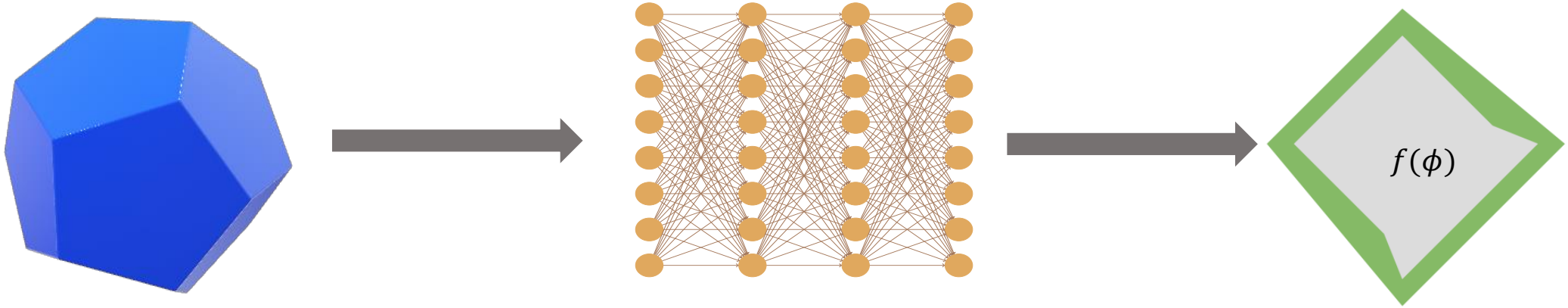
Precondition over network inputs ϕ



Network f

Postcondition over network outputs ψ

Formal verification of DNNs

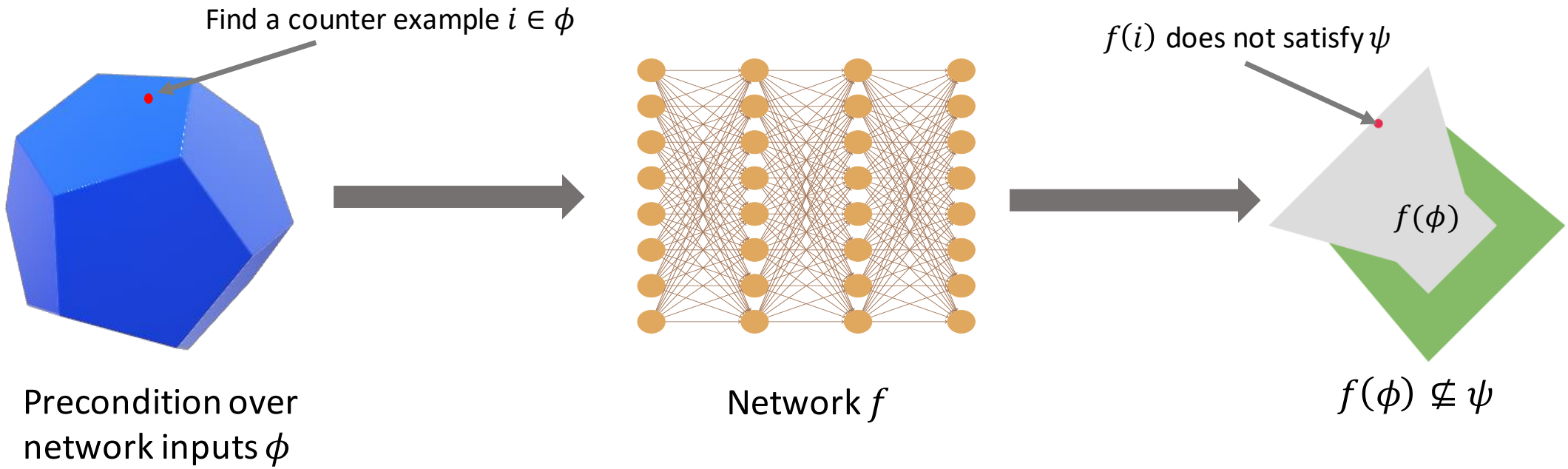


Precondition over network inputs ϕ

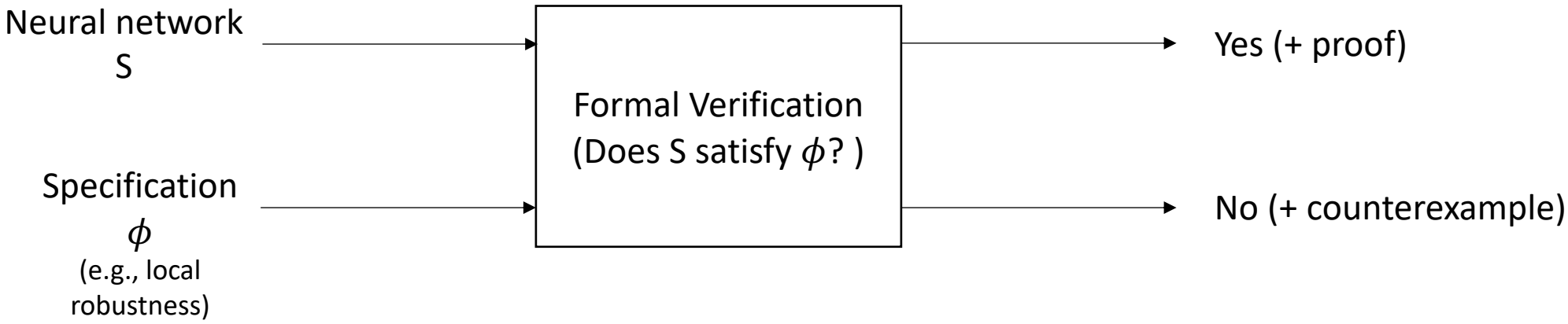
Network f

Prove $f(\phi) \subseteq \psi$

Formal verification of DNNs



Formal verification of DNNs



Soundness and completeness

Sound	Complete	Guarantee
Yes	Yes	If the verifier proves that the specification holds, then it indeed holds for the DNN. If the specification holds for the DNN, the verifier can prove it.
Yes	No	If the verifier proves that the specification holds, then it indeed holds for the DNN. However, even if the specification holds for the DNN, the verifier may not be able to prove it.
No	Yes	Even if the verifier proves that the specification holds, then it may not hold for the DNN. However, whenever the specification holds for the DNN, the verifier can prove it.
No	No	Even if the verifier proves that the specification holds, then it may not hold for the DNN. Moreover, even if the specification holds for the DNN, the verifier may not be able to prove it.

Desirable properties of a verifier:

- Soundness (always)
- As complete as possible
- Scalability

Formal verification of DNNs: Techniques

Incomplete	Abstract interpretation: Box, Zonotope, DeepPoly
Complete	Mixed Integer Linear Programming (MILP) SMT solvers (Reluplex)

Active area of research with annual competition: VNNComp
Current winner: alpha-beta crown